
Design and implementation of a network on chip-based simulator: a performance study

V. Sanju*, C. Koushika and R. Sharmili

School of Computing Science and Engineering,
VIT University,
Vellore – 632014, Tamilnadu, India
E-mail: sanjuv21@gmail.com
E-mail: koushika08@gmail.com
E-mail: rsharmili@gmail.com
*Corresponding author

Niranjan Chiplunkar

Department of Computing Science and Engineering,
NMAMIT Institute of Technology,
Nitte – 574110,
Karkala Taluk, Karnataka, India
E-mail: nchiplunkar@nitte.edu.in

M. Khalid

School of Computing Science and Engineering,
VIT University,
Vellore – 632014, Tamilnadu, India
E-mail: mkhalidbs@yahoo.com

Abstract: New trends and research outputs in VLSI fabrication technology have enabled integration over a billion transistors on a silicon die. To enable system development of this growing complexity, shared common bus structures were replaced by network on chip-based topologies such as 2D mesh, and torus. These systems used packet transfer techniques and provided communication mechanism among interconnected modules. These systems are realised as ASIC's or FPGA-based systems. Design of high performance systems requires an understanding of its internal characteristics, interaction among modules and working of these systems. These parameters are difficult to record on hardware platforms. To understand these process parameters, simulation studies are quite beneficial. This paper discusses the design and implementation of a simulator for network on chip-based systems. The topologies discussed are 2D mesh, torus and RiCoBiT: ring connected binary tree – a new structured and scalable topology for network on chip-based systems. The paper also studies the performance parameters of the same, along with some representative application and observations.

Keywords: network on chip; NoC; design; simulator; 2D mesh; torus; ring connected binary tree; RiCoBiT.

Reference to this paper should be made as follows: Sanju, V., Koushika, C., Sharmili, R., Chiplunkar, N. and Khalid, M. (2014) 'Design and implementation of a network on chip-based simulator: a performance study', *Int. J. Computational Science and Engineering*, Vol. 9, Nos. 1/2, pp.95–105.

Biographical notes: V. Sanju obtained his BE in Computer Science and Engineering from NMAMIT, Nitte, India in 2002, MTech in VLSI and Embedded Systems from Department of PG Studies, University Campus, VTU, Belgaum, India in 2005. Currently, he is pursuing his PhD with VIT University, Vellore, India. His area of interest includes network on chip architectures, CAD for VLSI, and embedded systems.

C. Koushika is currently an undergraduate student with School of Computing Science and Engineering, VIT University, Vellore, India.

R. Sharmili is currently an undergraduate student with School of Computing Science and Engineering, VIT University, Vellore, India.

Niranjan Chiplunkar obtained his BE in Electronics and Communication Engineering from National Institute of Engineering, Mysore in 1986, MTech in Computer Science and Engineering from Manipal Institute of Technology, Manipal in 1991 and PhD in Computer Science and Engineering from University of Mysore in 2001. His areas of interest include CAD for VLSI, computer networking and wireless body sensor networks. He has over 25 years of teaching and research experience. He is a member of IEEE, CSI, ISTE and Fellow of Institution of Engineers (India). He is currently working as the Vice Principal, Dean (Academics), HOD (CSE) with NMAMIT, Nitte, India.

M. Khalid is a Senior Professor at the School of Computing Science and Engineering (CS&E), VIT University, Vellore, India. He leads the School as a Dean/Director for four plus years. He obtained his PhD (1981), MTech (1977) and BSc Engineering (1974) degrees from IIT Bombay, IIT Delhi and AMU Aligarh, respectively. He has a long-standing teaching, research and administrative experience in CS&E at IIT Bombay (1981–1998), Jordan (1996–1998 on lien), Qatar (1998–2003), and Bahrain (2003–2005). His research areas include high speed computer networks, design and analysis of protocols, performance evaluation of computing systems and fast packet switch architecture.

1 Introduction

Semiconductor devices like transistors and CMOS have made revolutionary changes in design and implementation of digital systems. Initially, these devices were very large and integration of only a few devices in order of hundreds was possible. These devices were connected to form functional modules using common bus architecture. Newer techniques in IC process technology and mask generation has enabled fabrication of smaller devices which enabled more logic to be incorporated. This posed a serious threat in terms of performance while using common bus. Several protocols and bus architectures including multi bus structures and switching logic were introduced to increase the performance. These prevailed in design methods for last three decades by when the process technology was also further reducing. This initiated finding a replacement for common bus architecture. This led to the concept of network on chip (NoC).

NoC (Dally and Towles, 2011; Benini and De Micheli, 2002; Nurmi, 2005; Hemani et al., 2011) is a new paradigm in ASIC design. The method of system design is to implant principles of data communication network on a silicon chip. The different processing modules are placed in topologies like mesh, torus, tree, butterfly, octagon, etc. (Rantala et al., 2006). These modules communicate with each other by exchanging packet through the topology. This paper discusses design and implementation of a simulator for NoC-based systems. Mesh, torus and ring connected binary tree (RiCoBiT) are the topologies under consideration. Various performance parameters like packet latency and packet absorption time are studied under varying traffic loads and packet sizes. Also an application is implemented and tested on these topologies.

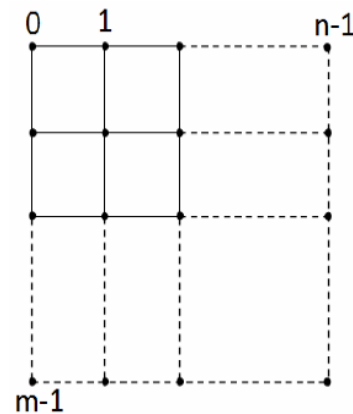
2 Topologies under consideration

This paper discusses design of a simulator for the following topologies.

2.1 Mesh

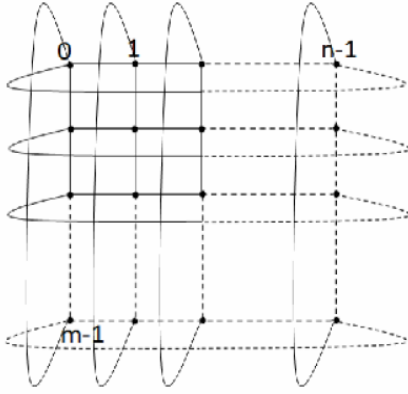
Figure 1 depicts 2D mesh topology which resembles a matrix (Rantala et al., 2006). The system of MN modules is specified by giving the number of rows (M) and columns (N) and are placed intersection of the row and the column as depicted in the figure. The modules are addressed relative to the row and columns.

Figure 1 Mesh topology



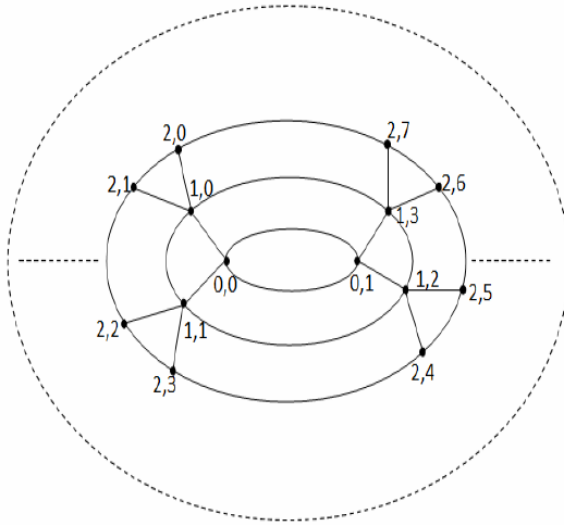
2.2 Torus

The topology is obtained from mesh topology by connected the end points as shown in Figure 2 (Rantala et al., 2006). The specification and addressing remain same as mesh topology.

Figure 2 Torus topology

2.3 RiCoBiT

It is obtained by interconnecting growing concentric rings with binary tree as depicting in Figure 3. Each ring in the topology has 2^R nodes where R is the ring number. The total number of nodes in the topology with R rings is $\Sigma 2^R$. The node addressed (R, Nr) represents a node numbered Nr in ring R . The numbering of the ring starts with inner ring as one and the node from zero to $2^R - 1$.

Figure 3 RiCoBiT topology

3 Node organisation and working

This section discusses the internals and working of a node in unidirectional and bidirectional form for the above mentioned topologies. The basic building block of the topology is a node which is composed of interfaces, through which it interacts with the adjacency node.

3.1 Organisation and working in unidirectional mode

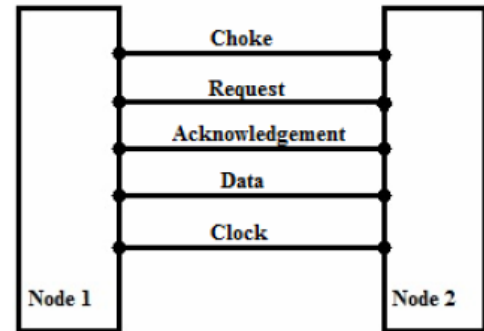
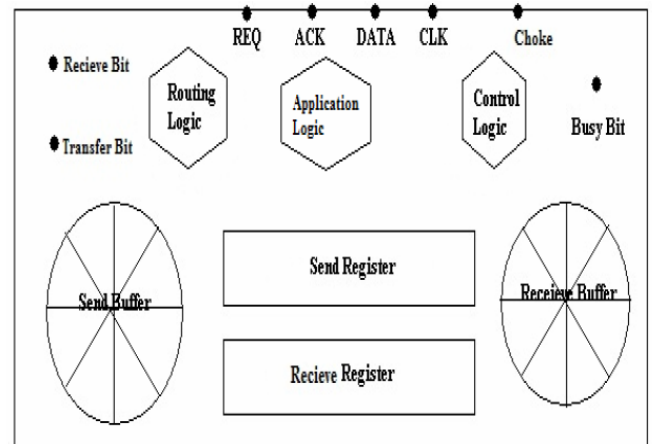
This subsection discusses node organisation and working in unidirectional mode where the interface can either receive or send packet any instant of time. In this case, the interface

would contain only single request acknowledgement, data, and clock pins for both transmission and reception. Figure 4 illustrates the interconnection between the two interfaces.

The description of the pins is as follows:

- Request (REQ): This pin is raised whenever there is a packet to be sent to the adjacent node. It also accepts the request signal raised by the adjacent interface for packet transfer.
- Acknowledgement (ACK): Upon the request signal, the acknowledgement pin is raised for packet transfer.
- Data (D): This pin is used to transfer packet between the interfaces serially.
- Clock (CLK): This pin is connected to the internal clock circuitry which is used for synchronisation between interfaces while packet transfers.
- Choke (C): This bit assists the interfaces to get a fair chance to transferring packets alternatively between them. During reception of data, if the send buffer at the receiving interface contains packets then the choke bit is set. The sending interface checks the choke bit and if it is set it allows the adjacent interface to set the request providing fairness

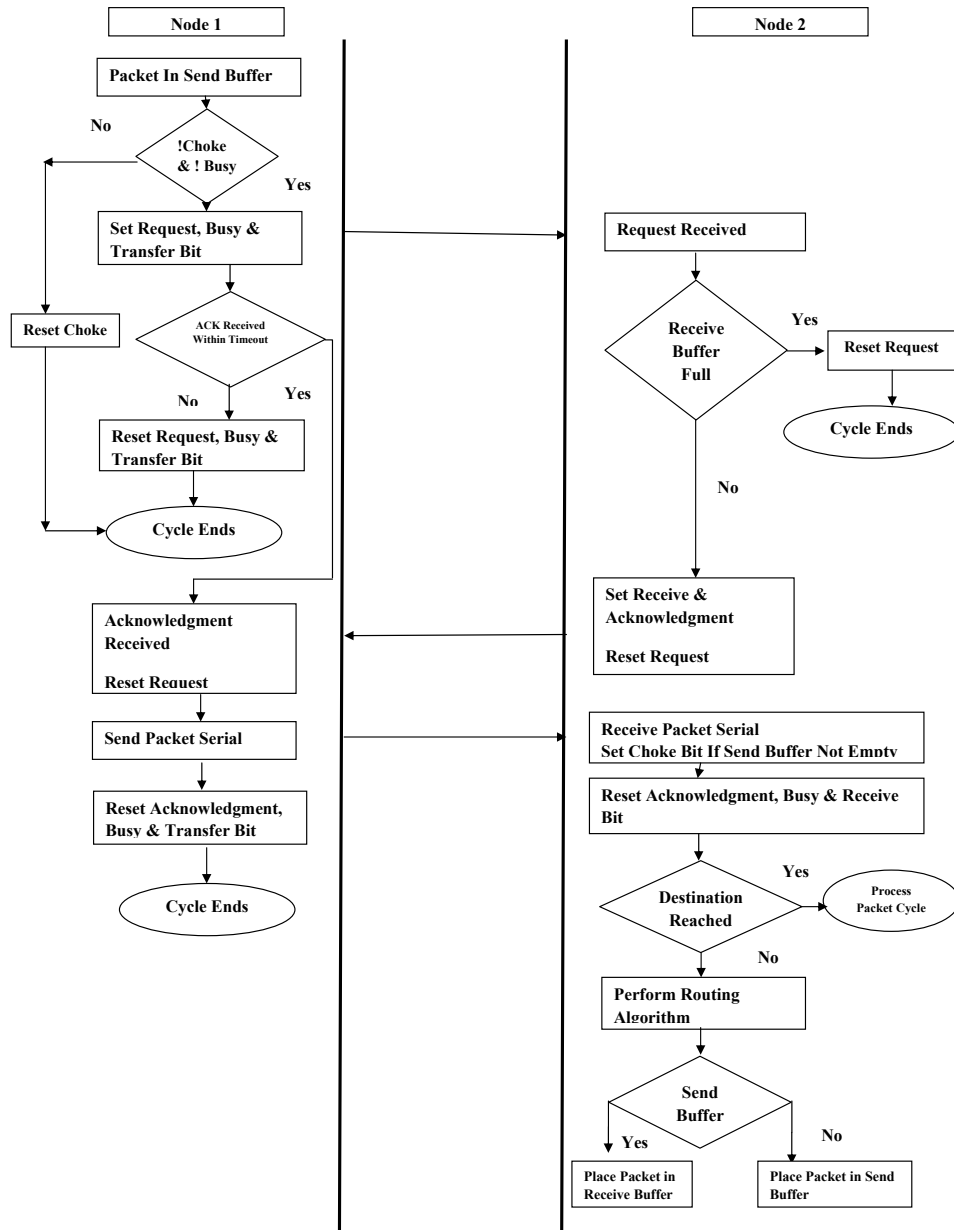
Figure 5 depicts the internals of the interface in this mode.

Figure 4 Interconnection of nodes in unidirectional mode**Figure 5** Internals of a node in unidirectional mode

As depicting in Figure 5, the interface consists of the following components apart from the pins discussed above:

- Receive register: This serial shift register assists the interface to receive the packet after acknowledgement is send.
- Send register: This serial shift register assists the interface in transmission of the packet once it receives the acknowledgement.
- Routing logic: This contains the routing algorithm which routes the packet from source to destination through the shortest route.
- Control logic: All the internal working is controlled by it by generating appropriate control signal.
- Busy bit: This status bit is set whenever the interface is sending or receiving data.
- Transfer bit: It refers that the interface is transmitting packet.
- Receive bit: This bit is set when the interface is given acknowledgement and is receiving packet.

Figure 6 Sequence diagram depicting the working in unidirectional mode



The working cycle starts when the node generates a packet and places it in appropriate send buffer. The control logic checks whether the choke and busy is set. If the choke is set, it resets the choke bit and the cycle continues. If both the signals are low then the sending interface raises request signal along with setting of the busy bit and transfer bit and waits for acknowledgement. A priority resolver is used to resolve conflicts if both interface raises its request pin simultaneously. If the request is not acknowledged by the adjacent interface within a timeout period then the request pin and status bits are reset. The acknowledgement is returned by the adjacent interface if there is a space in its receive buffer. This is accompanied by resetting of the request line at the interface and setting of the receive bit and the busy bit.

On reception of the acknowledgement at the sending interface, its request line is reset and the packet is placed in its send register which is sent serially through the data pin along with generation of internal clock. The packet is received serially in the receive register through the data pin. Choke bit is used to integrate fairness in transmission of packets alternately between interfaces. During reception of data, if the send buffer contains packets then the choke bit is set. The interface checks the choke bit and if it is set it allows the adjacent interface to set the request providing fairness. After the packet is fully received, the status bits are reset at both the interfaces. The packet is given to the application logic if the packet reaches destination else the routing algorithm is performed on the packet to determine the next interface along its destination. Now the control logic checks whether there is a space available in the send buffer along its destination. If there is a space it is transferred to send buffer else it is placed in the receive buffer at the receiving interface. Now the cycle continues till the packet reaches the destination where the packet is taken out of the system.

The sequence diagram (Figure 6) illustrates the working of two adjacent interfaces.

3.2 Organisation and working in bidirectional mode

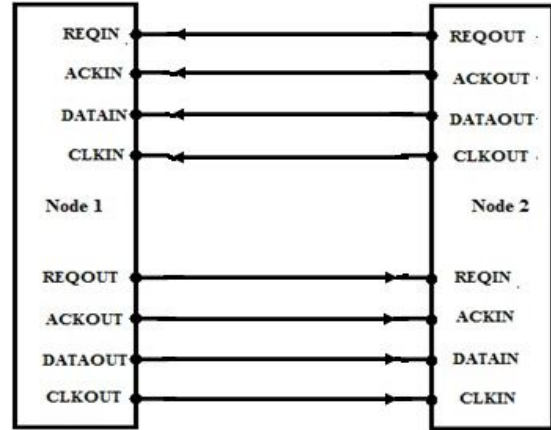
This subsection discusses node organisation and working in bidirectional mode where the interface can receive or send packet simultaneously. In this case, the interface would contain only separate request, acknowledgement, data and clock pins for both transmission and reception. Figure 7 illustrates the interconnection between the two interfaces.

The description of the pins is also follows:

- RequestIn (REQIN): This pin accepts the request from the adjacent interface. This is connected to REQOUT of the adjacent interface.
- AcknowledgementIn (ACKIN): It receives the acknowledgement signal from the adjacent interface and is connected to the ACKOUT.
- RequestOut(REQOUT)/AcknowledgementOut (ACKOUT): This pin works the same as request/acknowledgement pin of unidirectional mode.

- DataIn (DIN)/DataOut (DOU): These pins are used to transmit data bits.
- ClockIn (CLKIN)/ClockOut (CLKOUT): These are used to for synchronisation during packet transfer.

Figure 7 Interconnection of nodes in bidirectional mode



The operation starts the REQOUT pin raises itself when the send buffer has a packet to transfer. This is connected to the REQIN of the adjacent interface. In reply to the request, the ACKOUT is raised and then the packet is transferred. In this mode, parallel reception and transfer through an interface is possible. The operation of the buffers, registers, and routing logic remains the same.

The working of two adjacent interfaces is as illustrated in the sequence diagram (Figure 8). The working details are similar to that of unidirectional mode.

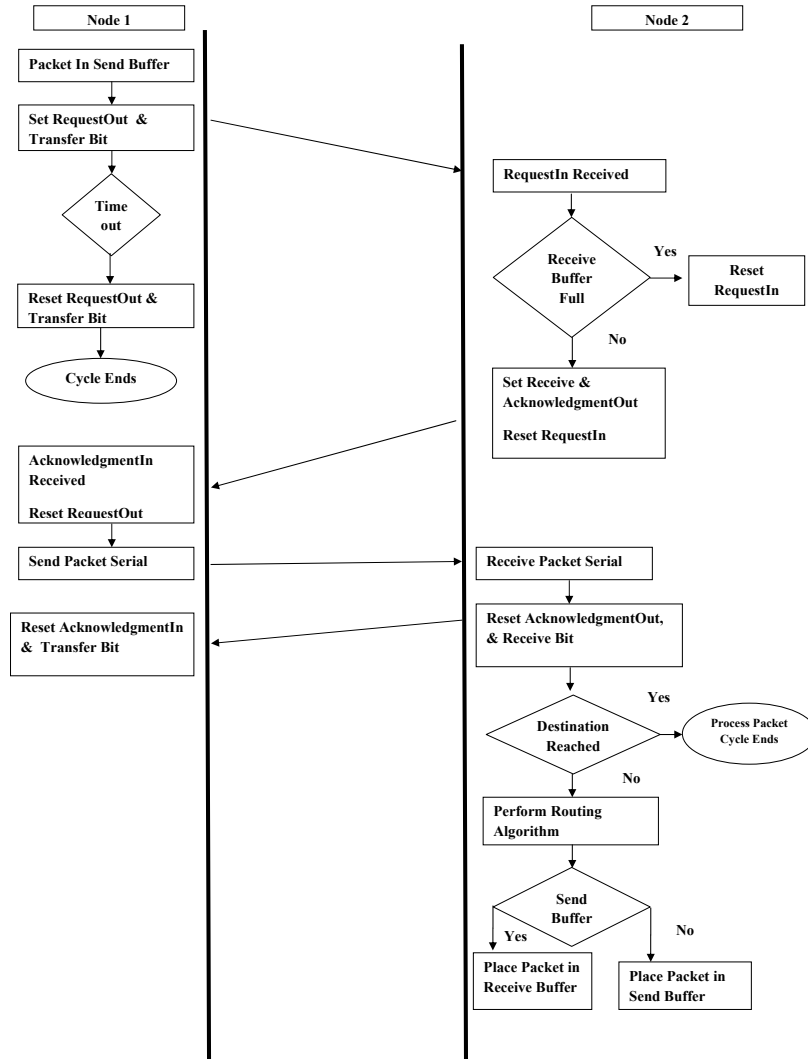
4 Simulator design and implementation

The above mentioned topology is designed using objects oriented concepts and is implemented using C++. The class definition below describes prototype of a node interface.

```

class Interface
{
    Bool REQ,ACK,DATA,CLK,CHOKE;
    //Pin Definition
    Bit Receive_Register[sizeof(packet)];
    // Register Definition
    Bit Send_Register[sizeof(packet)];
    Bit Receive_Buffer[sizeof(packet),n];
    //Buffer Definition
    Bit Send_Buffer[sizeof(packet),n];
    Bool Busy_Bit, Recieve_Bit, Transfer_Bit;
    // Status Bits
    Interface Routing_Algorithm(packet);
    // Routing Algorithm
    void Control_Logics();
    // Control Logic
    void Buffer_Operations();
    // Circular Buffer Operations
}

```

Figure 8 Sequence diagram depicting the working in bidirectional mode

This class definition serves as basic building block for a node which includes all the components of the interface as discussed above. The pins and the status bits are declared as one bit Boolean values. It also houses a bit array as big as the packet for the send and receive registers. The definition also contains a two dimensional bit array of packet size along with its operations which implements a circular queue of n location send/receive buffer. The routing logic represents appropriate topology routing. In case of mesh and torus, dimensional XY routing is used and RiCoBiT algorithm for the other.

The class definition below illustrates RiCoBiT node definition.

```

class RiCoBiT_Node
{
    Interface Left_Interface;
    Interface Right_Interface;
    Interface Top_Left_Interface;
    Interface Top_Right_Interface;
    Interface Bottom_Interface;
}
  
```

Node definitions for the other topologies are in similar lines as above. The above node definition is used to create topology of n nodes by creating a linked list of objects which will be discussed in subsequent section.

4.1 Data structures used

Various data structures were used in the implementation of the simulation. This section discusses the same.

4.1.1 Registers

These are implemented as normal array declaration

4.1.2 Status bits

These are normal one bit variable declaration

4.1.3 Circular buffer

The send and receive buffers are implemented as circular buffers. The size of the buffer is fixed by the user before the simulation starts. It is implemented as a linked list and

places the packet within itself. The usage of linked list enables implementation of priority.

4.1.4 Clock

This variable maintains the number of clock ticks during simulation. This variable is initialised as zero and is incremented by one in every iteration.

4.1.5 Packet

As discussed, the applications are realised by transferring packets from one node to another. The packet is realised as a structure as given below:

```
struct packet
{
    char source_address[];
    char dest_address[];
    char data[]
    int start_timer;
    int end_timer;
}
```

Whenever a node generates a packet, the variable of the above structure is created with the structure member start_timer assigned the current value of clock. Similarly, when the packet reaches the destination the clock value is assigned to the end_timer. All the performance parameters of the simulator are calculated using these timer values.

4.1.6 Packet generator

This is an integral part of the system. This module generates varying forms of traffic pattern to test the system. The output of the packet generator is stored in a double linked list. These packets are later filled in the send buffers of the interfaces at the source node which travels through the network and reaches the destination. Also whenever the node generates a packet, it is initially added to this list. The following packet generation modes are experimented in this design:

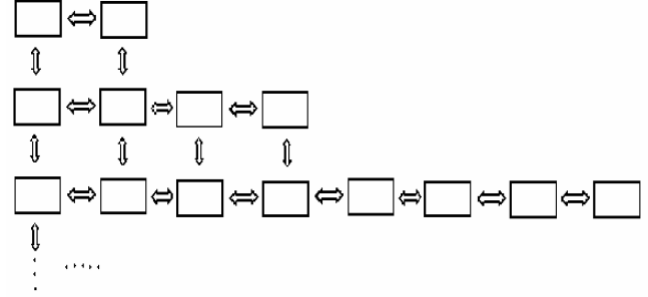
- nearest neighbour first from the node/interfaces
- nearest neighbour last from the node/interfaces
- longest neighbour first from the node/interfaces
- longest neighbour last from the node/interfaces
- alternate nearer and longer from the node/interfaces.

This generation can be based on a node, set of nodes or the entire network itself. The number of packets to be generated per node, packet sizes can also be specified. In addition to this, the user can create a comma separated file of packets to generate user defined traffic.

4.1.7 Topology

The topology is generated by creating an array of class instances in the form of a linked link as depicted in Figure 9. Member functions are incorporated for access to each node. The list starts with level one and has two nodes in case of RiCoBiT. This implementation is efficient in memory usage when compared to conventional 2D array.

Figure 9 Depicting the linked list for RiCoBiT topology



The simulation starts initialisation of the environment variables like number of nodes, data size in the packet, number of buffers, type of packet generation, which topology is to be used, output path and the file name. Once the variables are set, simulation starts and the results are generated in the form of list view and in file (*.txt format). The dataflow diagram (Figure 10) illustrates the different steps in the simulation process for unidirectional case.

Each iteration of the dataflow diagram represents one cycle of operation performed by the node interface. Every iteration the status of the pins, status registers, buffers are checked and appropriate steps are taken as per the working discussed in the previous section and in the dataflow diagram. It should be noted that the all interfaces of the topology works simultaneously.

The functionality is similar in case of bidirectional simulation.

5 Testing and verification

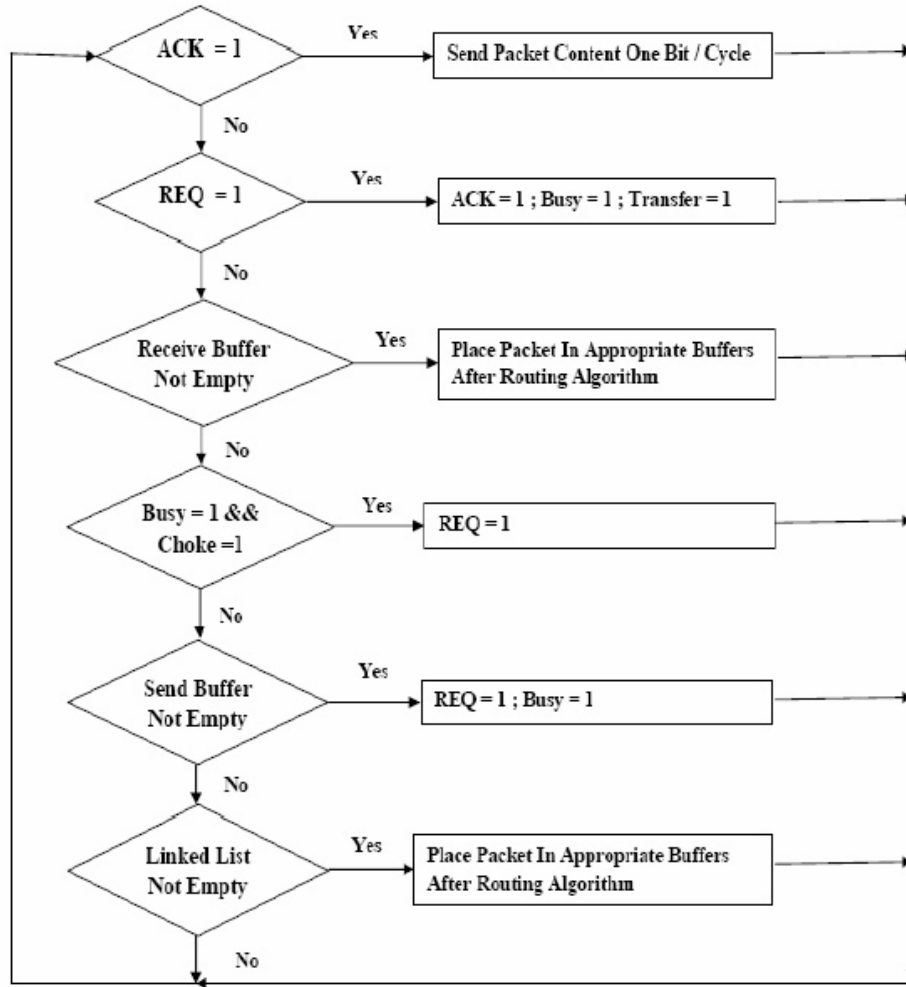
Various test cases generated and were given as input to the simulator for functional correctness. A few of the cases are discussed below.

5.1 Routing algorithms

The routing algorithm was tested with varying number of nodes. The packets were generated to all source destination node pairs and the distance between the nodes was tabulated. This was compared with that of all pair shortest path algorithm (Floyds). It was observed that the results generated are correct.

5.2 Buffers

Various inputs were given to verify buffer empty/buffer full and normal working of the buffers.

Figure 10 Data flow diagram for unidirectional mode of operation

Separate test cases were generated to test shifting operation of the send/receive registers.

5.3 Overall working

Various test cases depicting the different working of the system was derived and fed as input to the simulator. The output generated in various sub system of the simulator in each cycle was recorded in a text file. This also verifies that the simulator is functionally correct. The snapshot (Figure 11) depicts the test case generation for parallel working of all interfaces in a RiCoBiT node.

6 Results

Various simulation runs were conducted to test the strength of the above mentioned topology. Performance parameters like maximum packet latency, average packet latency and packet absorption rate were recorded under varying number of buffers for different packet size. The following graphs depict the above mentioned parameters for packet of eight bits data using longest neighbour first from the interfaces.

The graph in Figure 12 is recorded by varying the number of nodes and by sending one packet between the nodes of the longest distance. From the graph, it is evident that the latencies are always high in case of mesh. But it is almost equal for torus and proposed architecture whenever the number of nodes is less. As the topology scales up the difference in latencies becomes quite noticeable and the trend continues. This implies that the packet takes more time to reach the destination thereby decreasing the performance in torus and mesh.

Now the topologies are subjected to increasing traffic loads by steadily increasing the number of nodes generating the packets. The simulation started with one node generating packets to all the other nodes in the system with an increase by factor of 10%. During the simulation runs parameters like the maximum time taken for any packet to reach its destination, average packet latency and the time taken for the system to absorb all the packets were recorded. This is referred as maximum cycle count, average cycle count and overall packet delivery time respectively. Figures 13, 14 and 15 depict these parameters for one node sending to all other nodes with 40% buffer filled.

Figure 11 Sample output screen for parallel working of the interfaces in RiCoBiT

Simulation Results For RiCoBiT For Parallel Working Of Interfaces

Level 9

The packet size is 77

Total no. of packets created 50

Data pushed into the Mod[5][16] top right buffer – packet – 49-1

Data pushed into the Mod[5][16] top right buffer – packet – 48-0

Data pushed into the Mod[5][16] top left buffer – packet – 46-1

Data pushed into the Mod[5][16] top left buffer – packet – 45-0

Data pushed into the Mod[5][16] bottom buffer – packet – 43-1

Data pushed into the Mod[5][16] bottom buffer – packet – 42-0

Data pushed into the Mod[5][16] right buffer – packet – 38-1

Data pushed into the Mod[5][16] right buffer – packet – 37-0

Data pushed into the Mod[5][16] left buffer – packet – 25-1

Data pushed into the Mod[5][16] left buffer – packet – 24-0

Initialisation Done

ITERATION NO. 1

Req sent to right interface of mod 5 17 for packet 25 0

Req sent to left interface of mod 5 15 for packet 38 0

Req sent to top right interface of mod 4 8 for packet 43 0

Req sent to bottom interface of mod 6 33 for packet 46 0

Req sent to bottom interface of mod 6 32 for packet 49 0

ITERATION NO. 2

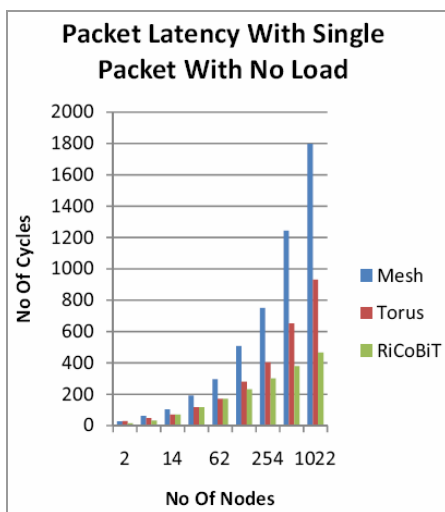
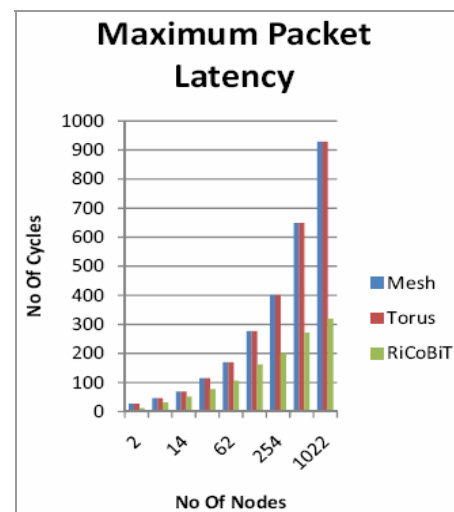
Acknowledgement sent to bottom of 5 16 for packet 43

Acknowledgement sent to right of 5 16 for packet 38

Acknowledgement sent to left of 5 16 for packet 25

Acknowledgement sent to top right of 5 16 for packet 49

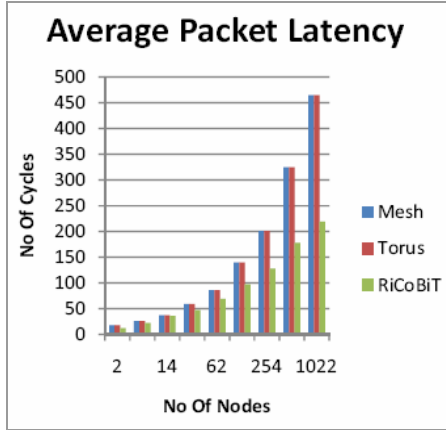
Acknowledgement sent to top left of 5 16 for packet 46

Figure 12 Packet latency with single packet with no load (see online version for colours)**Figure 13** Maximum packet absorption time in unidirectional mode (see online version for colours)

Note: One node sending packets to all nodes

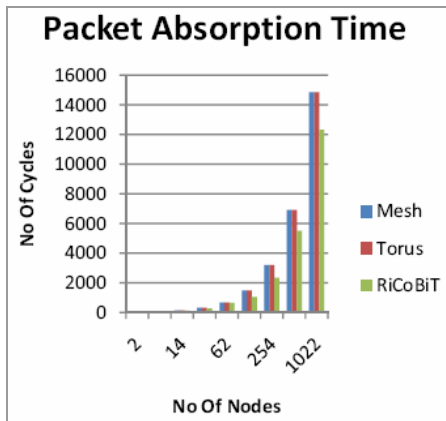
From the graphs, it is evident that an application implemented using RiCoBiT would perform better than mesh and torus. Also, it is observed that performance factor is not that affected with scalability in case of RiCoBiT.

Figure 14 Average packet absorption time in bidirectional mode (see online version for colours)



Note: One node sending packets to all nodes

Figure 15 Overall packet delivery time in unidirectional mode (see online version for colours)



Note: One node sending packets to all nodes

7 Application simulation – matrix multiplier

As part of the simulation study, an application called matrix multiplication was implemented on the above mentioned topology to test the strength. The application was implemented and tested in both unidirectional and bidirectional mode on a 32×32 mesh/torus and level nine for RiCoBiT. The user can start the application by providing the order of the matrix and input data. The computation of the product is carried out by passing packet to different nodes such that each node gets data to compute on cell of the matrix. Table 1 tabulates the results for the same for both unidirectional and bidirectional mode.

From Table 1, it is evident that RiCoBiT performs better than mesh and torus. It is observed that RiCoBiT gives 20% to 25% more performance than the others.

Table 1 Results of matrix multiplication

Matrix dimension	Unidirectional			Bidirectional		
	Mesh	Torus	RiCoBiT	Mesh	Torus	RiCoBiT
2×2	77	77	77	77	77	77
3×3	231	231	185	231	231	185
4×4	433	433	433	433	433	379
5×5	745	745	621	745	745	621
6×6	1,261	1,261	1,121	1,261	1,261	911
7×7	1,873	1,873	1,561	1,873	1,873	1,327
8×8	2,881	2,881	2,237	2,817	2,817	1,721
9×9	3,921	3,921	3,197	3,841	3,841	2,257
10×10	5,301	5,301	4,161	5,201	5,201	3,017

Table 2 Packet size in RiCoBiT

Number of nodes/level	Packet size	
	Ring and node number	Only node number
2/1	10	10
6/2	14	14
14/3	18	16
30/4	20	18
62/5	24	20
126/6	26	22
254/7	28	24
510/8	30	26
1,022/9	34	28
2,046/10	36	30

8 Effect of node addressing on performance/throughput in RiCoBiT

From various simulation runs, it is observed that node addressing schemes play an important role in performance of the system. The transmission time of a packet between the interfaces depends on the number of bits the packet contains. This implies that if the transmission time would increase as the packet size grows thereby decreasing the performance.

Let us consider two addressing schemes for RiCoBiT.

- Using ring and node number: In this case, the node is referred by the ring number to which it belongs and the position within the ring. This is as shown in figure. In this case the size of a packet with source, destination addresses and K bits data can be expressed as $2 * (\log_2 R + R) + K$ where R is the maximum number of rings in the topology.
- Using only node number: In this case, the nodes are addressed using continuous numbers starting from the inner ring as shown in figure. In this case, the size of a packet with source, destination addresses and K bits data can be expressed as $2 * (\log_2 N) + K$ where N is the number of nodes in the topology. It is observed that the packet size is reduced when using only node numbers. Table 2 illustrates the same for a byte data. This trend increases as the topology grows.

Table 3 Performance comparison of RiCoBiT with two different addressing schemes

Number of nodes/level	Ring and node number			Only node number		
	Overall absorption time	Maximum time	Average time	Overall absorption time	Maximum time	Average time
2/1	15	12	12	15	12	12
6/2	54	35	24	54	35	24
14/3	136	58	40	121	52	36
30/4	325	93	57	268	77	47
62/5	753	126	82	632	106	69
126/6	1,218	190	114	1,037	162	97
254/7	2,700	233	149	2,327	201	128
510/8	9,672	331	217	5,500	271	178
1,022/9	20,338	264	264	12,325	320	219

9 Conclusions

The implementation of the simulation has helped in studying the characteristics and internal working of the topologies. From the above presented simulation runs and different experiments conducted on the topologies, RiCoBiT fares better than mesh and torus. The path traversed in the case of RiCoBiT is much smaller than mesh and torus which is an added advantage in case of RiCoBiT. Also the presence of an extra interface increases the performance/throughput factor for RiCoBiT.

From the scalability point of view, it is observed that the performance parameter scales well in case of RiCoBiT. The parameters increase substantially in mesh and torus but only marginally in RiCoBiT. Also matrix multiplier application also supports the observation that RiCoBiT is better. Observations in term of addressing schemes are also an important outcome of the study.

References

- Dally, W.J. and Towles, B. (2011) 'Route packets not wires: on chip interconnection network', *DAC 2001*, June.
- Rantala, V., Lehtonen, T. and Plosila, J. (2006) 'Network on chip routing algorithms', TUCS Technical Report No. 779, August.
- Benini, L. and De Micheli, G. (2002) 'Networks on chips: a new SoC paradigm', *IEEE Computer*, Vol. 35, No. 1.
- Nurmi, J. (2005) 'Network on chip: a new paradigm for system on chip design', *Proceedings 2005 International Symposium on System on Chip*, 15–17 November.
- Hemani, A., Jantsch, A., Kumar, A., Postula, A., Berg, J., Millberg, M. and Lindqvist, D. (2011) 'Network on a chip: an architecture for billion transistor era', *DAC 2001*.